

The original elliptic interface is connected

Actual group transport, literal outputs and complete signed quartic lifts

An explicit additive equivalence for generalized Weierstrass square completion, native equality with returned coordinate lists, and nonsquare or wrong-sign lift elimination.

16

reusable theorem declarations
audited

9

fully connected worked packets

3

previous interface gaps closed

Actual original-model halving and exact literal rational output semantics within certificate budgets. Generic JSON parser refinement, efficient Sturm proofs and global rank remain separate.

Source, corpus receipts and reproduction commands accompany the repository snapshot.

The three completed bridges

This 6 October 2026 continuation closes the three explicit gaps in the preceding native-halves release. Square completion now has a proved additive equivalence between Mathlib's actual rational point groups. Every new packet proves that its native original-model fibre has exactly the literal coordinate list returned in the JSON result, and that its original target has the supplied literal coordinates. A complete signed quartic-lift evaluator now handles rational abscissae whose cubic values are nonsquares, as well as the two possible ordinate signs and their actual doubling test.

The public operation remains `native_halves_certificate`, with registered elliptic objects exposing `native_halves`. The packet schema is now `pp-native-halves-certificate/2`. It includes `original_points` as well as `completed_points`. The native theorems `original_json_list_checked`, `completed_json_list_checked`, `original_target_checked` and `literal_fibre_complete` tie these literal rational lists to actual point-group semantics. The returned JSON parser implementation is not itself formalized; the source contains the exact mathematical literals represented by the packet, and its source hash is retained. Generic efficient Sturm verification also remains separate from the bounded native signed-divisor root route.

Sixteen reusable theorem declarations and the generated packet declarations are audited. Nine worked packets cover infinity, ordinary and branch-target root-free obstructions, nonsquare quartic lifts, single- and four-element cosets, generalized coefficients, and both target signs in a forced quartic route. The complete repository archive preserves the parallel elliptic subgroup and flavor changes integrated before publication.

Explicit transport of the actual point group

Let W be the generalized Weierstrass equation

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6.$$

Put $A=a_2+a_1^2/4$, $B=a_4+a_1a_3/2$ and $C=a_6+a_3^2/4$. The completed model is $Y^2=x^3+Ax^2+Bx+C$, where $Y=y+(a_1x+a_3)/2$. The new module `EllipticSquareTransport` proves equality of the discriminants, exact equation equivalence, and nonsingularity equivalence under the nonzero discriminant hypothesis. It constructs both actual point maps, including infinity, and proves they are inverses.

The essential new step is preservation of addition. The proof handles infinity directly. A vertical line in the original coordinates remains vertical after the shift, because the reflection relation transforms exactly. For nonopposite points with equal x-coordinates, subtracting their two curve equations factors into the ordinate difference times the reflection factor. Since they are not opposite, the reflection factor cannot vanish, so the ordinates agree and the tangent branch applies.

Both tangent and secant slopes transform by the same law: the completed slope is the original slope plus $a_1/2$. The proof treats denominator nonvanishing explicitly. Substitution into the actual addition formulas shows equality of the sum's abscissa and the

prescribed shift of its ordinate. These coordinate identities are then applied to Mathlib's actual point addition, giving `forward_add`. Together with the two inverse maps, this constructs a genuine additive equivalence `equiv`.

Every integer multiplication fibre therefore transports through the map, not only doubling or two-torsion. The reusable fibre theorem expresses that preservation. `list_transport` moves a complete fibre list through an arbitrary additive equivalence. The group transport has no supplied matrix, rank, bounded-search completeness or interpreter-correctness premise. Smoothness is the natural elliptic-curve hypothesis, and each concrete packet proves it for the original model.

A coefficientwise model equality connects the generic completed model to the exact coefficients used by the preceding root and halving certificates. Identity casting along that equality preserves coordinates. The packet's `originalEquiv` combines this cast with the inverse additive equivalence. Its `originalHalves` is the inverse transport of the native completed fibre, and `original_halves_complete` quantifies over every actual point of the original generalized curve.

Literal output equality reaches the user coordinates

The native `coordinates` function sends infinity to none and an actual affine point to its rational coordinate pair. Its injectivity theorem states that these encodings distinguish actual points. This is a typed mathematical encoding; canonical rational strings in JSON represent the same rational literals in the emitted source.

The producer uses an explicit ordered list of the proved rational cubic roots, rather than relying on an unspecified ordering of a finite set. A coset packet evaluates its native group list in that declared order. Infinity packets use the same ordered branch list. Complete quartic packets retain the root order, consider the nonnegative square root first and its negative second, and keep only actual doubles of the target. Numeric original-model output is emitted in this native list order. Completeness is independent of ordering, while exact list equality requires it to be declared.

`completed_json_list_checked` proves equality between the map of native completed points to coordinates and the literal completed coordinate list in the packet. It reduces concrete actual point additions and doubling expressions, including their exceptional cases, with proved rational arithmetic. For quartic lifts, finite square-root equalities are kernel reduced first, then the signed point and filter expressions are checked. This avoids leaving an unevaluated negative integer square root or an opaque point predicate in a purported literal result.

The reusable `coordinates_backward` theorem gives the original ordinate exactly: $y = Y - (a_1x + a_3)/2$. Applying it to the complete native list and the checked completed literal list yields `original_json_list_checked`. The original target receives an analogous equality. Finally `literal_fibre_complete` combines complete group-list membership with coordinate injectivity, proving that an arbitrary actual original-model point doubles to the checked target exactly when its coordinates occur in the literal returned list.

For the generalized fixture $[1, -1/4, 1, -1/2, -9/4]$, the original anchor is (3,3), while the completed anchor is (3,5). Both lists are checked in Lean and the transported target is checked against the supplied original target. This prevents the completed-model theorem from being silently presented as a theorem about unshifted user coordinates.

Nonsquares and both signs are complete

The new module `EllipticQuarticLifts` defines the two possible lifts of each rational abscissa x . Let s be Mathlib's rational square-root function applied to x^3+Ax^2+Bx+C . If s^2 equals that cubic value, the list contains the actual points (x,s) and $(x,-s)$, with their curve equations and nonsingularity proved. Otherwise it is empty.

Mathlib's `Rat.exists_mul_self` theorem proves that the square-root equality is equivalent to existence of a rational square root. From any actual point (x,y) , the curve equation establishes the equality. Factoring y^2-s^2 then proves $y=s$ or $y=-s$. The reusable `mem_lifts` theorem therefore includes every actual affine point in the finite signed list for its abscissa. No assertion by Python's square-root routine is a mathematical premise.

For an affine target (u,v) , the complete halving quartic is

$$x^4 - 4u*x^3 + (-2B-4uA)*x^2 + (-8C-4uB)*x + B^2-4AC-4uC = 0.$$

Every actual half must have an abscissa in the complete rational root list. Infinity and branch points cannot double to an affine target. The native `fibreList` flat-maps all quartic roots through the signed lift list, then filters by the actual point equality $[2]Q=P$. The reusable `fibre_complete` theorem proves completeness of that filtered list. It handles nonsquares, zero square roots, reflection signs and any candidate discarded by the actual doubling condition. A quartic root is never mistaken for a rational elliptic point.

On $y^2=x^3+x$, the target (0,0) has halving quartic $(x^2-1)^2$ and rational roots -1 and 1. Their cubic values are -2 and 2, so both lifts are absent over the rationals. The returned original-model fibre is empty and its literal equality is proved. This was outside the previous root-free quartic gate and is now covered natively.

On $y^2=x^3-7$, (2,1) doubles to (32,-181), while (2,-1) doubles to (32,181). Both target signs have the same quartic root list containing 2 and the same two signed candidates. The actual doubling filter retains the correct sign in each case. Both full native original-model fibres and their exact literal lists are checked. These packets force `route="quartic"`, so the signed-lift evaluator is exercised even though an anchor could have supplied the coset route.

The default `route="auto"` retains the efficient anchor route when a half is found, and uses complete signed quartic lifts when no anchor is found. Infinity remains the complete two-torsion kernel. The signed-divisor degree, coordinate-bit and work budgets remain explicit. Large scaled quartics may exceed the budget and be rejected without returning a partial completeness claim. The mathematical lift and transport theorems themselves quantify over arbitrary rational coordinates and complete supplied root lists.

Reproduction, evidence and the remaining frontier

`make native-halves-refinement-receipts` regenerates nine packets, nine cold service responses and the combined audit. `make native-halves-refinement-lean` builds the two native modules, compiles every generated packet, prints the axioms of the reused and new completeness declarations, and runs lint. The dedicated tests cover original and completed coordinate distinctions, source hashes, both signed quartic targets, nonsquare lifts, native coset ordering, direct and registered operations, invalid routes and work-budget rejection. Existing halving, elliptic arithmetic, division, subgroup and native-bridge tests are rerun around the changed interfaces.

The stored validation receipt records exact source hashes, declaration names and test totals. The whole historical repository build is not represented as refreshed by these focused checks. The delivered archive is checked byte for byte against the published Git tree, and the PDF is checked against the committed monograph. Earlier release receipts remain historical evidence; this edition supersedes their explicit original-transport, literal-list and nonsquare-lift limitations for newly emitted version-two packets.

The remaining general interface work is the JSON parser implementation proof and an efficient native Sturm route for larger scaled polynomials. Global Mordell-Weil rank, saturation and integral-point bounds remain additional mathematical programs. The parallel subgroup machinery can now use an actual original-model halving fibre whose coordinates are bound to the native result, while tripling and higher composite fibres still require their corresponding native point-law and division-polynomial interfaces.